

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Instituto de Ciências Exatas e Informática

Engenharia de Software

LABORATÓRIO DE EXPERIMENTAÇÃO DE SOFTWARE

Análise Comparativa da Presença de Vulnerabilidades Conhecidas (CVE) em Dependências de Projetos Node.js Hospedados no GitHub com e sem Ferramentas Automatizadas

Autores:

Gustavo Delfino Guimarães

Matheus Caetano Rocha

Belo Horizonte, maio de 2026

Sumário

Sumário.....	2
1. Introdução.....	2
2. Questões de Pesquisa.....	3
3. Metodologia.....	3
3.1 Etapa 1 – Busca e Seleção dos Repositórios.....	3
3.2 Etapa 2 – Extração e Análise dos Repositórios.....	3
3.3 Etapa 3 – Consolidação dos Dados.....	4
3.4 Etapa 4 – Análises Estatísticas.....	4
3.5 Métricas Utilizadas.....	4
4. Resultados.....	5
4.1 RQ1 – Frequência de Dependências Vulneráveis.....	5
4.2 RQ2 – Nível de Severidade e Distribuição Proporcional de Risco.....	6
4.4 RQ4 – Diferenças entre Categorias Funcionais de Bots.....	10
5. Discussão dos Resultados.....	12
5.1 Hipóteses e Confirmações.....	12
5.2 Achados Principais.....	12
5.3 Limitações do Estudo.....	13
6. Conclusão.....	13
7. Referências.....	13

1. Introdução

O desenvolvimento moderno de software é fortemente baseado no uso de bibliotecas e dependências externas, prática que favorece a reutilização de código, redução do tempo de desenvolvimento e rápida incorporação de funcionalidades. No ecossistema Node.js, tal abordagem é amplamente viabilizada pelo gerenciador de pacotes npm, responsável por disponibilizar um vasto conjunto de bibliotecas open source.

Embora esse modelo apresenta benefícios significativos, ele também introduz desafios relacionados à segurança de software. Dependências podem conter vulnerabilidades conhecidas registradas pelo sistema Common Vulnerabilities and Exposures (CVE), que podem ser propagadas para aplicações que as utilizam. A presença de bibliotecas vulneráveis ou desatualizadas expõe projetos a riscos de segurança e dificulta o gerenciamento adequado das dependências, especialmente em projetos com árvores extensas e múltiplos níveis de dependência.

Este trabalho investiga a incidência de vulnerabilidades conhecidas em dependências de projetos Node.js hospedados no GitHub, avaliando frequência, severidade e distribuição dessas vulnerabilidades, bem como o impacto do uso de ferramentas automatizadas (bots) em sua redução.

2. Questões de Pesquisa

O estudo busca responder às seguintes questões:

#	Questão de Pesquisa
RQ1	Qual é a frequência de dependências vulneráveis em projetos Node.js hospedados no GitHub?
RQ2	Quais são os níveis de severidade e a distribuição proporcional das vulnerabilidades identificadas?
RQ3	A utilização de bots automatizados está associada a uma menor incidência de dependências vulneráveis?
RQ4	Existem diferenças entre categorias de bots quanto ao impacto na redução de vulnerabilidades?

3. Metodologia

A metodologia deste estudo foi estruturada em um fluxo sequencial de quatro etapas, definidas de forma a segmentar as atividades de busca, coleta, processamento e análise dos dados.

3.1 Etapa 1 – Busca e Seleção dos Repositórios

Foi realizada a busca de repositórios públicos hospedados no GitHub utilizando a GitHub Search API. Foram considerados projetos cuja linguagem principal é JavaScript ou TypeScript e que utilizam Node.js como ecossistema de desenvolvimento. A presença do arquivo package.json foi utilizada como critério de identificação de projetos Node.js. Repositórios que não possuem esse arquivo foram descartados da amostra. Os repositórios elegíveis foram clonados localmente para análises subsequentes. Ao final desta etapa, a amostra foi composta por 9.982 repositórios.

3.2 Etapa 2 – Extração e Análise dos Repositórios

A segunda etapa consistiu na execução dos scripts de análise desenvolvidos para extração das informações necessárias ao estudo. Inicialmente, foi realizada a identificação do gerenciador de dependências por meio dos arquivos lockfile, incluindo package-lock.json, yarn.lock e pnpm-lock.yaml. Posteriormente, as dependências efetivamente resolvidas foram extraídas para análise.

Em seguida, os pacotes identificados foram comparados com bases públicas de vulnerabilidades, particularmente Open Source Vulnerabilities (OSV) e National Vulnerability Database (NVD), permitindo identificar vulnerabilidades conhecidas associadas às dependências por meio de identificadores CVE e classificá-las conforme o Common Vulnerability Scoring System (CVSS).

Paralelamente, foi realizada a detecção de ferramentas automatizadas presentes nos repositórios, incluindo Dependabot, Renovate, Depfu, Snyk, OWASP Dependency Check, Sonatype, Frogbot, Mend, CodeQL e npm audit.

3.3 Etapa 3 – Consolidação dos Dados

Após a execução dos scripts de análise, os dados extraídos foram consolidados em uma base única. Durante esse processo, foram verificadas inconsistências e registros incompletos. Repositórios cuja extração não foi concluída com sucesso foram descartados da análise estatística final.

3.4 Etapa 4 – Análises Estatísticas

A última etapa consistiu na realização das análises estatísticas com o objetivo de investigar padrões e possíveis associações entre vulnerabilidades, severidade e adoção de ferramentas automatizadas. As análises utilizaram estatísticas descritivas, gráficos de distribuição, boxplots e métricas comparativas entre grupos.

3.5 Métricas Utilizadas

Métrica	Descrição
M1	Distribuição dos repositórios por ferramenta automatizada detectada
M2	Total de dependências diretas por grupo de ferramenta
M3/M2	Proporção de dependências vulneráveis vs. seguras
M4	Percentual de repositórios com ≥ 1 vulnerabilidade por grupo
M5	Percentual de dependências vulneráveis com fix disponível
M6 (CVSS)	Distribuição de CVEs por nível de severidade
M7	Proporção alto+crítico no perfil de severidade

4. Resultados

Esta seção apresenta os resultados obtidos a partir da análise do conjunto de dados composto por 9.982 repositórios públicos do ecossistema Node.js, coletados em maio de 2026.

4.1 RQ1 – Frequência de Dependências Vulneráveis

A primeira questão de pesquisa quantifica a extensão da presença de vulnerabilidades conhecidas nos projetos. Dos 9.982 repositórios analisados, 5.709 (57,19%) contêm pelo menos uma vulnerabilidade detectada, enquanto 4.273 (42,81%) encontram-se livres de ameaças. No total, foram contabilizadas 63.493 dependências vulneráveis atreladas a 137.987 CVEs associados.

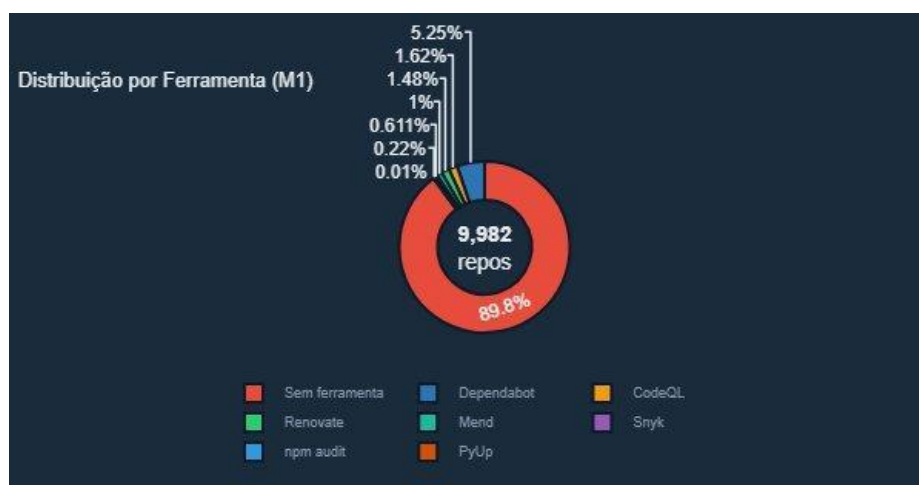


Gráfico 1 – Distribuição dos repositórios por ferramenta automatizada (M1). 89,8% dos repositórios não utilizam nenhuma ferramenta.

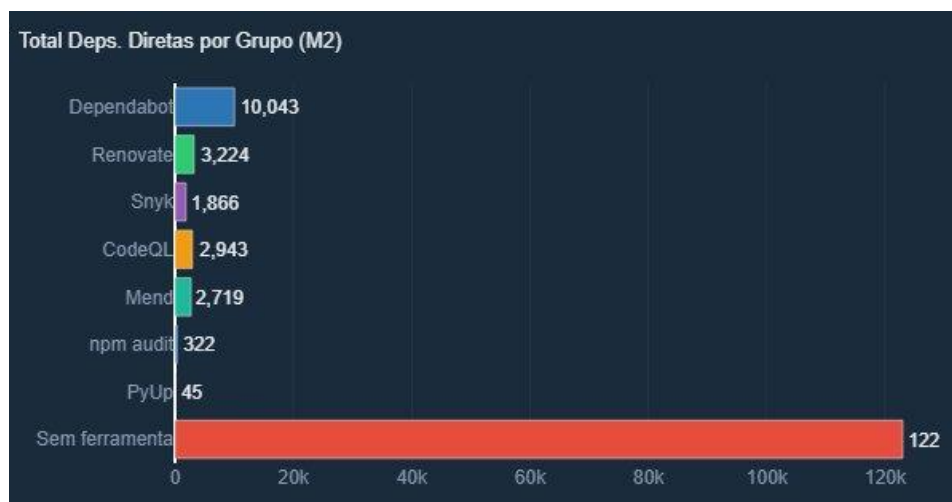


Gráfico 2 – Total de dependências diretas por grupo de ferramenta (M2).

A tabela abaixo expõe a disparidade entre o grupo de projetos saudáveis e o grupo afetado:

Métrica (Média por Grupo)	Com Vulnerabilidade	Sem Vulnerabilidade
Dependências diretas / repo	16,74	11,37
Versões resolvidas / repo	120,83	3,10
Dependências vulneráveis / repo	11,12	0,00
CVEs / repo	24,17	0,00

Projetos vulneráveis carregam, em média, uma árvore de dependências significativamente mais profunda (120,83 versões resolvidas contra apenas 3,10 do grupo sem vulnerabilidades). Essa disparidade indica que o crescimento e o adensamento da árvore de subdependências expandem a superfície de ataque de forma acentuada, gerando uma média de 24,17 CVEs por repositório afetado.

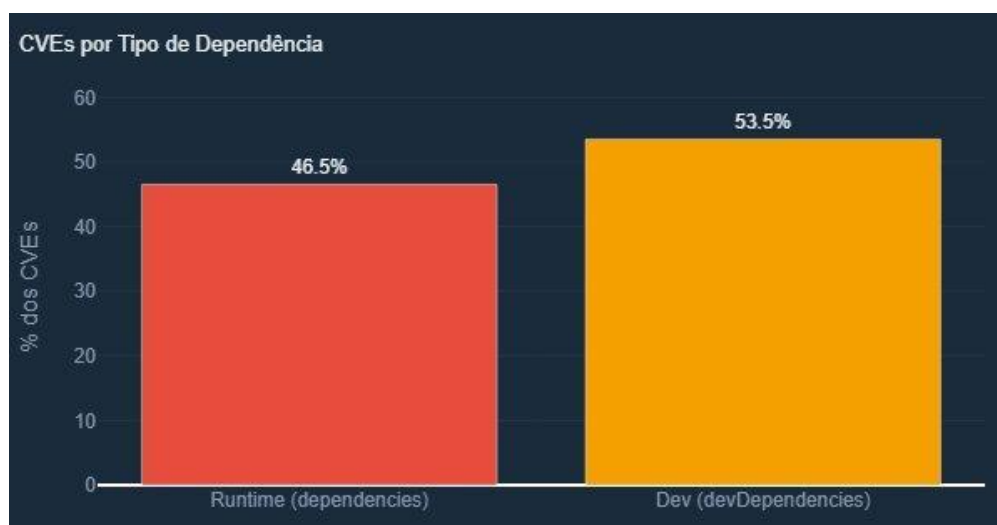


Gráfico 3 – CVEs por tipo de dependência: 53,5% em devDependencies e 46,5% em dependências de runtime.

4.2 RQ2 – Nível de Severidade e Distribuição Proporcional de Risco

A segunda questão analisa a gravidade das vulnerabilidades identificadas de acordo com as métricas do CVSS. No volume acumulado de todos os repositórios coletados, foram mapeados 137.987 CVEs.

Nível CVSS	Qtd. de CVEs	% do Total
MEDIUM	58.697	42,54%
HIGH	50.619	36,68%
LOW	10.725	7,77%

Nível CVSS	Qtd. de CVEs	% do Total
CRITICAL	10.722	7,77%
UNKNOWN	7.224	5,24%
TOTAL HIGH + CRITICAL	61.341	44,45%

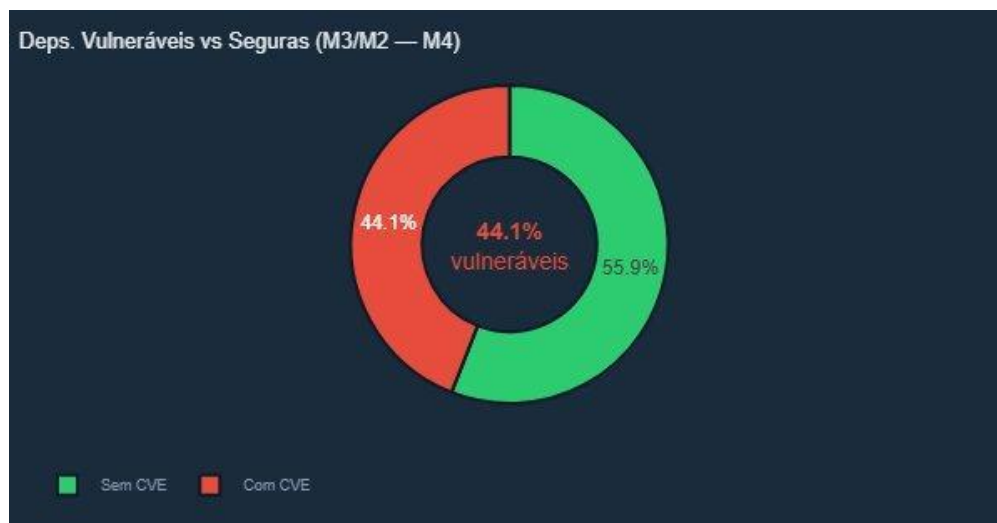


Gráfico 4 – Dependências vulneráveis vs. seguras (M3/M2 → M4): 44,1% das dependências apresentam ao menos um CVE.

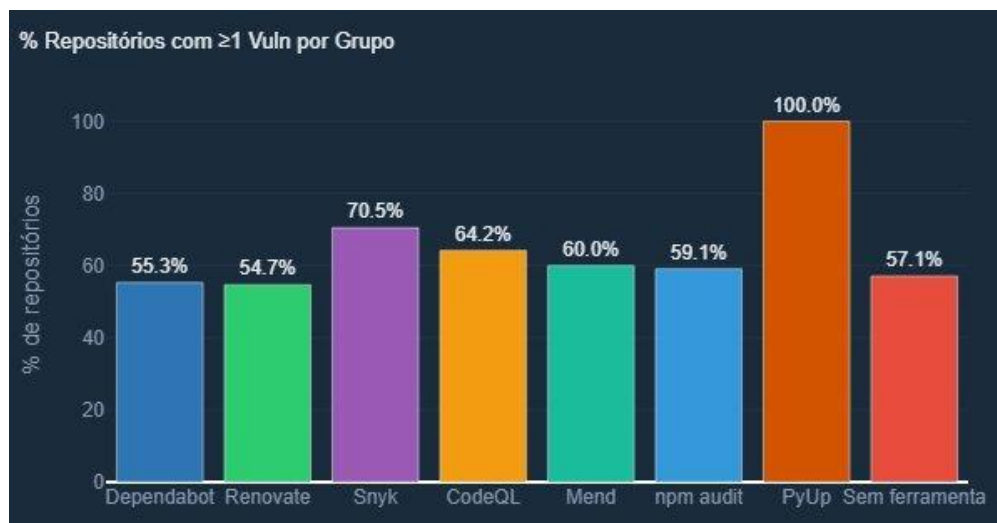


Gráfico 5 – % de repositórios com ≥ 1 vulnerabilidade por grupo de ferramenta.

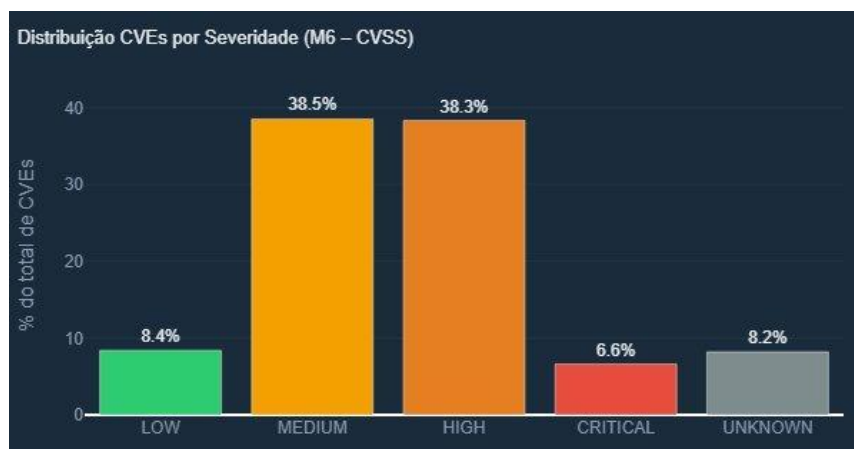


Gráfico 6 – Distribuição de CVEs por severidade (M6 – CVSS): concentração em MEDIUM (38,5%) e HIGH (38,3%).



Gráfico 7 – Proporção por nível CVSS (M6). Alto + Crítico somam 45,0% (M7).



Gráfico 8 – Severidade por grupo de ferramenta (M6 por grupo). Dependabot e Renovate apresentam as menores proporções de CVEs críticos.

4.3 RQ3 – Impacto de Ferramentas Automatizadas na Incidência de Riscos

A terceira questão busca verificar se a adoção de ao menos uma ferramenta automatizada correlaciona-se com a diminuição da proporção de projetos vulneráveis.

Grupo Analítico	n (repos)	% Repos com ≥ 1 Vuln	Volume Absoluto (aprox.)
Sem bot	8.964	57,08%	5.117
Com bot (1+)	1.018	58,15%	592

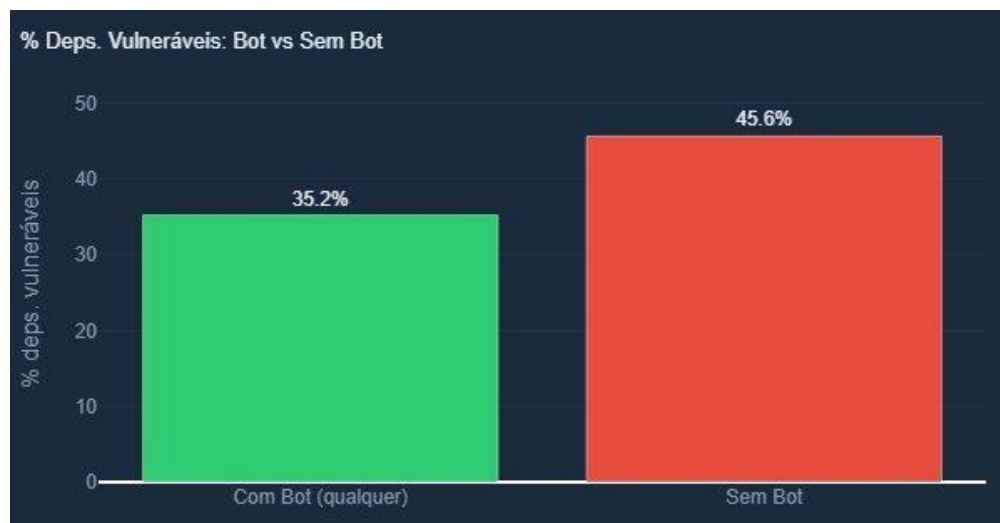


Gráfico 9 – % de dependências vulneráveis: Com Bot (35,2%) vs. Sem Bot (45,6%).

Os dados empíricos indicam que não há evidência de redução na incidência absoluta de vulnerabilidades decorrente da mera adoção de automações. O grupo com ferramentas apresentou taxa discretamente superior ($\sim +1,07$ pp). Contudo, quando analisada a densidade normalizada de vulnerabilidades por 100 versões resolvidas, os projetos sem automação apresentam o pior índice (9,79), enquanto os ecossistemas com ferramentas de atualização e auditoria ativa operam com taxas significativamente reduzidas (3,61 a 5,41).

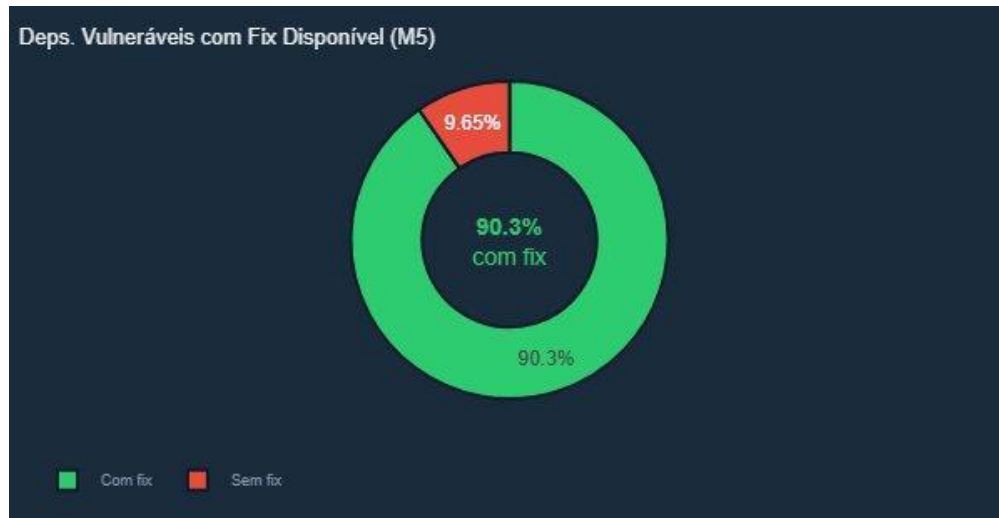


Gráfico 10 – Dependências vulneráveis com fix disponível (M5): 90,3% possuem correção disponível.

4.4 RQ4 – Diferenças entre Categorias Funcionais de Bots

A quarta questão investiga se diferentes categorias funcionais de bots apresentam comportamentos distintos em relação à incidência de vulnerabilidades. As ferramentas foram agrupadas conforme seu objetivo principal:

Categoria	Ferramentas	Deps vuln/100	CVEs/100	Incidência%
none (sem bot)	—	9,79	21,53	57,1%
update	Renovate, PyUp	5,41	10,49	55,0%
update security	Dependabot	4,48	8,76	55,3%
audit automation	npm audit	3,61	6,94	59,1%
security automation	CodeQL	7,93	16,08	64,2%
security remediation	Mend	7,96	15,11	60,0%
security scanner	Snyk	8,78	18,86	70,5%

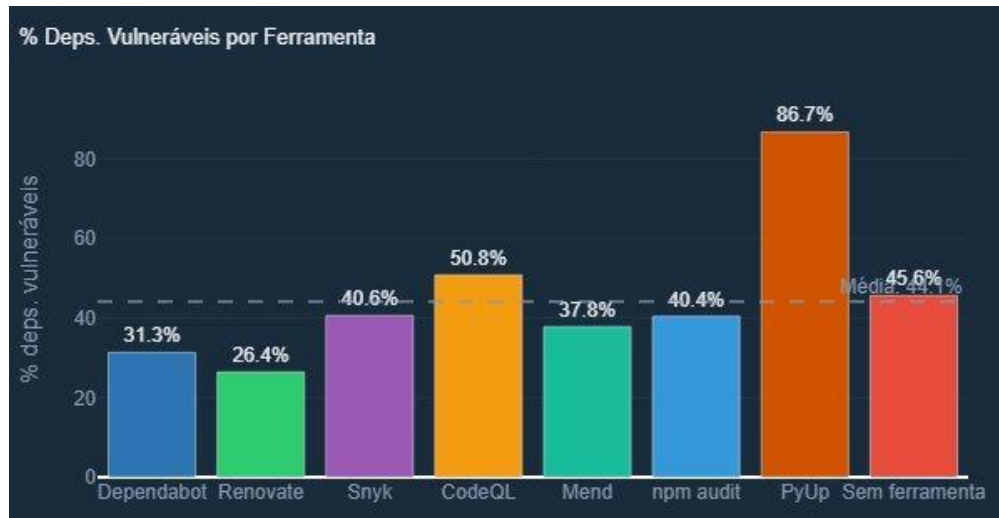


Gráfico 11 – % de dependências vulneráveis por ferramenta. Renovate (26,4%) e Dependabot (31,3%) abaixo da média (44,1%).

Os resultados revelam um comportamento bifurcado: ferramentas voltadas para atualização de pacotes (update e update security) exibem as menores taxas de projetos vulneráveis (~55%). Em contrapartida, ecossistemas com security scanner (Snyk) registram a maior taxa de incidência (70,49%), fenômeno que reflete o viés de seleção — plataformas de monitoramento robustas tendem a ser ativadas em projetos de maior porte.

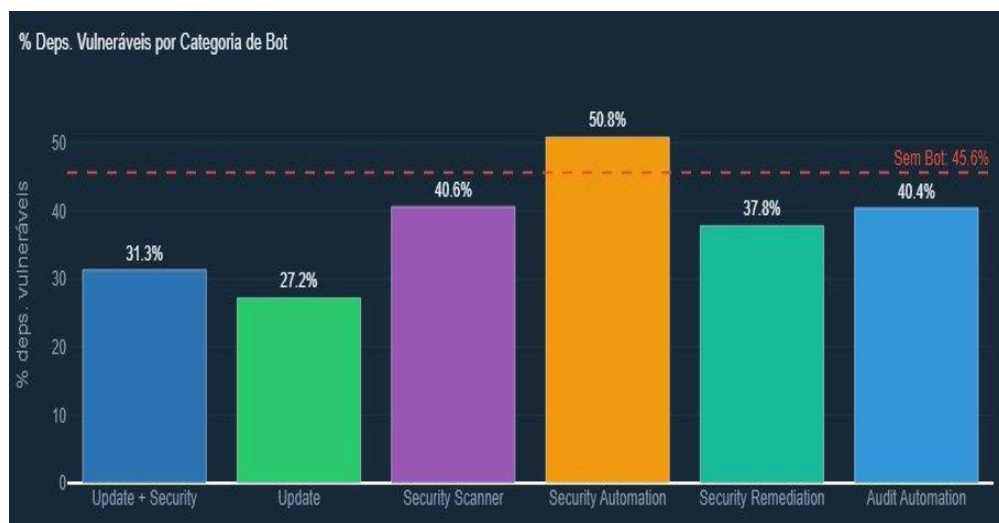


Gráfico 12 – % de dependências vulneráveis por categoria de bot. Categorias 'Update + Security' (31,3%) e 'Update' (27,2%) abaixo da linha de referência Sem Bot (45,6%).

5. Discussão dos Resultados

5.1 Hipóteses e Confirmações

Hipótese	Resultado
H1 (RQ1): Parcela significativa dos repositórios apresenta deps. vulneráveis	✓ CONFIRMADA – 57,19% dos repos possuem ≥ 1 CVE
H1 (RQ2): Predominância de vulnerabilidades de maior severidade	✓ CONFIRMADA – HIGH+CRITICAL somam 44,45% dos CVEs
H1 (RQ3): Uso de bots associado a menor incidência	✗ NÃO CONFIRMADA (incidência absoluta) / ✓ PARCIAL (menor severidade e densidade)
H1 (RQ4): Diferenças entre categorias de bots	✓ CONFIRMADA – categorias 'update' e 'update security' apresentam menor densidade

5.2 Achados Principais

Os resultados revelam que mais da metade dos projetos Node.js analisados (57,19%) apresentam ao menos uma vulnerabilidade CVE nas suas dependências. Esse dado reforça que a gestão de segurança de dependências é um problema crítico e generalizado no ecossistema JavaScript/Node.js.

A distribuição de severidade demonstra que quase metade (44,45%) das vulnerabilidades mapeadas são classificadas como HIGH ou CRITICAL, demandando atenção imediata. O dado de que 90,3% das dependências vulneráveis possuem um fix disponível (M5) sugere que grande parte dos riscos é tecnicamente mitigável, mas permanece não corrigida.

A análise comparativa entre grupos com e sem bots revela um resultado paradoxal: projetos com ferramentas automatizadas não apresentam menos repositórios vulneráveis em termos absolutos. Esse fenômeno pode ser explicado pelo viés de seleção — projetos corporativos de maior escala e complexidade tendem a adotar ferramentas de governança de dependências, e esses mesmos projetos possuem árvores de dependências mais densas, o que aumenta a superfície de ataque.

Quando analisada a densidade normalizada (vulnerabilidades por 100 versões resolvidas), o impacto positivo das ferramentas torna-se evidente: ferramentas de atualização como Dependabot e Renovate reduzem a proporção de falhas de 9,79 para 4,48 e 5,41, respectivamente. A combinação codeql + dependabot alcança 4,03 — uma redução de mais de 58% em relação ao grupo sem automação.

A distribuição de CVEs por tipo de dependência (Gráfico 3) revelou que 53,5% dos CVEs estão em devDependencies, o que indica que vulnerabilidades em ferramentas de desenvolvimento (não apenas em código de produção) representam um vetor de risco relevante e frequentemente negligenciado.

5.3 Limitações do Estudo

O estudo apresenta algumas limitações que devem ser consideradas na interpretação dos resultados. O viés de seleção é o principal fator de confundimento: projetos que adotam bots tendem a ser mais complexos e com mais dependências, o que eleva a incidência absoluta de vulnerabilidades mesmo com melhor gestão. Além disso, a análise considera apenas dependências diretas e resolvidas via lockfile, podendo subestimar vulnerabilidades transitivas em repositórios sem lockfile atualizado. Por fim, a categorização funcional dos bots é aproximada, pois algumas ferramentas oferecem funcionalidades híbridas.

6. Conclusão

Este estudo analisou 9.982 repositórios Node.js do GitHub, mapeando 137.987 CVEs distribuídos em 63.493 dependências vulneráveis. Os resultados confirmam que vulnerabilidades em dependências são prevalentes no ecossistema Node.js, com 57,19% dos projetos afetados e concentração significativa em severidades HIGH e CRITICAL.

Ferramentas automatizadas de atualização de dependências — especialmente Dependabot e Renovate — demonstraram associação positiva com a redução da densidade proporcional de vulnerabilidades, mesmo que não eliminem completamente o risco em nível binário de projeto. A combinação de múltiplas ferramentas (especialmente codeql + dependabot) apresentou o melhor desempenho na higiene da árvore de dependências.

As evidências empíricas sugerem que a adoção de ferramentas automatizadas de atualização e segurança é uma estratégia eficaz para redução do risco relativo de vulnerabilidades, devendo ser incentivada como prática de segurança em projetos Node.js.

7. Referências

HE, R.; HE, H.; ZHANG, Y.; ZHOU, M. Automating dependency updates in practice: An exploratory study on github dependabot. *IEEE Transactions on Software Engineering*, v. 49, n. 8, p. 4004–4022, 2023.

LIU, C.; CHEN, S.; FAN, L.; CHEN, B.; LIU, Y.; PENG, X. Demystifying the vulnerability propagation and its evolution via dependency trees in the npm ecosystem. In: *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, New York, NY, USA: ACM, 2022. p. 672–684.

YITAGESU, S.; XING, Z.; ZHANG, X.; FENG, Z.; BI, T.; HAN, L.; LI, X. Systematic literature review on software security vulnerability information extraction. *ACM Transactions on Software Engineering and Methodology*, v. 35, n. 4, p. 1–52, 2026.

